

Mesure et optimisation de la somme des sinus en Go

INF2007 – Programmation Avancée · TN4

Melissa Moya · Semaine 10

1. Approche et structure du programme

Le programme calcule la somme des sinus d'un tableau de 1 000 000 d'éléments (entiers ou flottants) via le flag `--type`. Deux fonctions spécialisées (`computeSineSumInt` / `computeSineSumFloat`) contiennent la boucle de calcul ; `computeSineSum` dispatche via `interface{}` avec un surcoût négligeable (~1 ns) face à `math.Sin` (~20–40 ns). Les tableaux sont générés avec `rand.NewSource(42)` pour la reproductibilité. Le biais de `rand.Intn(1001)` pour les plages non puissances de deux — décrit par la méthode de Lemire (`toIntervalUnbiased`) — est négligeable : l'objectif est d'obtenir des données représentatives, non une distribution parfaitement uniforme. La fonction `run()` retourne un struct `RunResult` sans affichage ; `main()` gère seule l'affichage (séparation logique/affichage).

2. Résultats des benchmarks

22 sous-benchmarks (11 paliers × 2 types) exécutés avec `go test -bench=. -benchmem -count=6` sur un Intel i5-10300H à 2,50 GHz (Windows/amd64, 8 threads), analysés avec `benchstat` (médianes + IC 95 %). `b.ResetTimer()` exclut le setup ; `b.ReportAllocs()` mesure la mémoire. Les tableaux sont pré-générés hors de `b.N` pour ne pas biaiser la mesure. Résultat : **0 B/op, 0 allocs/op** — la différence de performance provient du calcul, non d'allocations cachées. 13 tests unitaires, couverture 100 %.

Tableau 1 – Temps de calcul par type et pourcentage du tableau (médianes `benchstat`, 6 exécutions, i5-10300H)

% du tableau	Éléments	Int (ms)	Float (ms)	Ratio
1 %	10 000	0.44	0.24	1.86×
10 %	100 000	4.09	2.11	1.94×
20 %	200 000	8.11	4.24	1.91×
30 %	300 000	11.83	7.79	1.52×
40 %	400 000	15.52	8.99	1.73×
50 %	500 000	19.28	11.98	1.61×
60 %	600 000	22.98	13.61	1.69×
70 %	700 000	26.58	14.69	1.81×
80 %	800 000	30.94	16.82	1.84×
90 %	900 000	34.78	18.96	1.83×
100 %	1 000 000	38.71	20.93	1.85×

3. Analyse des résultats

Complexité et écart Int/Float

Les flottants sont **1,5 à 1,9× plus rapides**. De 50 % à 100 %, le temps double presque exactement (19,28 → 38,71 ms pour Int ; 11,98 → 20,93 ms pour Float), confirmant la complexité **O(n)**. L'écart provient de la conversion `float64(v)` à chaque itération, et du coût de réduction d'argument de `math.Sin` pour les grands entiers (ex. : `sin(500)` réduit ~79 tours modulo 2π), alors que les flottants dans $[0, 1)$ se situent déjà dans le domaine principal. Les IC à ± 1 % aux paliers 90–100 % confirment la stabilité des mesures.

Hiérarchie mémoire — L1/L2/L3

L'i5-10300H dispose de L1 = 256 KB, L2 = 1 MB, L3 = 8 MB. Les petits paliers (1–10 %, 80–800 KB) tiennent entièrement en L2 ; le palier 30 % (~2,4 MB) dépasse le L2 et sollicite le L3 ; le tableau complet ($1\,000\,000 \times 8$ octets = **8 MB**) atteint exactement la limite du L3, limitant les accès par la bande passante. Les benchmarks Float étant plus courts, la latence mémoire y représente une fraction relative plus grande — ce qui amplifie la variabilité observée aux paliers 30 % et 70 % (± 7 % vs ± 1 % pour Int/100 %).

Mécanismes micro-architecturaux

Inlining. `go build -gcflags=-m` confirme que `math.Sin` est inliné aux lignes 40 et 49 de `sinesum.go`, éliminant le surcoût d'appel (sauvegarde de registres, changement de frame) et rendant le coût par itération strictement déterministe. **Dépendance de données.** La boucle `sum += math.Sin(v)` crée une dépendance séquentielle sur l'accumulateur — identique à l'exemple `prefixSum` du ch. 6 — qui empêche le pipeline de paralléliser les additions. Les appels `math.Sin(v)` pour des `v` différents sont indépendants et pipelinables (exécution dans le désordre), mais ce gain est limité par la latence intrinsèque de `math.Sin`. Utiliser plusieurs accumulateurs fusionnés en fin de calcul constituerait la principale piste d'optimisation.

4. Applications numériques

Temps par sinus (médiane benchstat à 100 %) : Int = $38\,710\,000\text{ ns} \div 1\,000\,000 = \mathbf{38,7\text{ ns}}$; Float = $20\,930\,000\text{ ns} \div 1\,000\,000 = \mathbf{20,9\text{ ns}}$.

Q1 — Distance parcourue par la lumière pendant un sinus

$c = 299\,792\,458\text{ m/s}$

Type	Temps	Calcul	Distance
Int	38,7 ns	$299\,792\,458 \times 38,7 / 10^9$	≈ 11,6 m
Float	20,9 ns	$299\,792\,458 \times 20,9 / 10^9$	≈ 6,3 m

La lumière parcourt entre 6 et 12 mètres pendant un seul calcul de sinus.

Q2 — Nombre de sinus par tick à 120 fps

Durée d'un tick : $1 / 120 = 8\,333\,333\text{ ns}$.

Type	Temps/sinus	Calcul	Sinus/tick
Int	38,7 ns	$8\,333\,333 / 38,7$	≈ 215 333
Float	20,9 ns	$8\,333\,333 / 20,9$	≈ 398 726

Environ 215 000 (Int) ou 399 000 (Float) sinus par frame à 120 fps — marge confortable pour des effets visuels simples.

5. Conclusion

Les flottants sont **1,85× plus rapides** que les entiers, principalement en raison de la conversion `float64(v)` et de la réduction d'argument de `math.Sin` pour les grands entiers. La complexité $O(n)$ est confirmée empiriquement, et aucune allocation mémoire n'est détectée (0 B/op). L'utilisation de `testing.B` avec `benchstat` sur 6 exécutions produit des médianes fiables ($\pm 1\%$ aux paliers 90–100 %), bien supérieures à une mesure par `time.Since`. La vérification par `go build -gcflags=-m` révèle l'inlining de `math.Sin`, rendant les mesures représentatives du pur coût de calcul.

Références

- Documentation Go : `math/rand`, `testing`, `flag` — <https://pkg.go.dev>
- Lemire, D. (2019). Fast random integer generation in an interval. *ACM Transactions on Modeling and Computer Simulation*.
- Documentation Mermaid XY Chart — <https://mermaid.js.org/syntax/xyChart.html>
- Dépôt GitHub : github.com/moyamelissa/Advanced-Programming/tree/main/TN4
- GitHub Copilot utilisé comme assistant avec vérification systématique des suggestions.